## Objetivos

- Analizar diferentes mecanismos de **sincronización de procesos.**

## Referencias

[1] Tanenbaum, Bos – Modern Operating Systems - Prentice Hall; 4 edition (March 10, 2014) - ISBN-10: 013359162X
[2] Douglas Comer - Operating System Design - The Xinu Approach. CRC Press, 2015. ISBN : 9781498712439
[3] Silberschatz, Galvin, Gagne - Operating Systems Concepts - John Wiley & Sons; 10 edition (2018) – ISBN 978-1-119-32091-3

## Software y Hardware

Linux y Xinu. La versión de Xinu que utilizamos es para arquitectura PC (x86). Ejecutamos el sistema operativo Xinu en una máquina virtual llamada QEMU, que emula una PC básica.

El trabajo puede realizarse sobre las máquinas de los laboratorios (RECOMENDADO).

Quienes tengan Linux en sus casas, podrían intentar instalar todo lo necesario y llevarlo a cabo ahí también. Una tercera posibilidad es el acceso remoto RDP comentado en la web de la materia.

**Ejercicio 1.** *Sincronización entre procesos.*

a. Agregar este programa al shell de Xinu (recuerde cambiar el nombre de main por el nombre que seleccione para el programa):

```
#include <xinu.h>

void    produce(void), consume(void);

int32   n = 0;        /* Global variables are shared by all processes   */

/*------------------------------------------------------------------------
 *  main  -  Example of unsynchronized producer and consumer processes
 *------------------------------------------------------------------------
 */
void    main(void)
{
        resume( create(consume, 1024, 20, "cons", 0) );
        resume( create(produce, 1024, 20, "prod", 0) );
}

/*------------------------------------------------------------------------
 * produce  -  Increment n 2000 times and exit
 *------------------------------------------------------------------------
 */
void    produce(void)
{
        int32   i;

        for( i=1 ; i<=2000 ; i++ )
                n++;
}

/*------------------------------------------------------------------------
 * consume  -  Print n 2000 times and exit
 *------------------------------------------------------------------------
 */
void    consume(void)
{
        int32   i;

        for( i=1 ; i<=2000 ; i++ )
                printf("The value of n is %d \n", n);
}
```

b. Este programa es un típico programa compuesto de 3 procesos, main, productor, consumidor. Los procesos productor y consumidor se "comunican" a través de una variable compartida. El productor genera valores, y el consumidor los muestra en pantalla. Ejecutar el programa. Describir qué sucede. ¿El programa funciona como el programador pretendió?

**Ejercicio 2**. A partir de la bibliografía indicada y del modelo de semáforos utilizado en XINU, se solicita realizar un estudio del mecanismo de sincronización provisto por el sistema.

*a. Modelo estructural del semáforo en XINU (ayuda: pág 128):*

- Describir la estructura de un semáforo indicando sus componentes.
- Explicar qué información representa cada uno de estos componentes dentro del sistema.
- ¿Qué significa en XINU la tabla de semáforos global?

*b. Primitivas en XINU.* Analizar el código fuentede las funciones del sistema: ***semcreate() , wait() , signal()*** (están también descriptas en el pdf de este tema en la sección Apuntes)**.** Para cada una, se solicita:

***i. Interfaz y uso***

- Indicar el propósito de la primitiva.
- Especificar los parámetros de entrada y el valor de retorno.
- Describir en qué situaciones debe utilizarse.

***ii. Comportamiento operacional***

-Describir cómo se modifica el valor del contador del semáforo al ejecutar la primitiva.
-Indicar las condiciones bajo las cuales un proceso se bloquea o continúa su ejecución en wait().
-Indicar las condiciones bajo las cuales un proceso es desbloqueado en signal().
-Explicar el rol de la cola de procesos asociada al semáforo (squeue).
-Relacionar el valor del contador del semáforo con la cantidad de procesos en espera.

c. Modifique el programa del punto anterior, utilizando algún mecanismo provisto por el sistema operativo para la sincronización de procesos, de manera que el programa funcione como el programador pretendía.

**Ejercicio 3.** *Implementación de recursos lógicos.*

a. Agregar este programa al shell de Xinu.

```
/* mut.c - mut, operar, incrementar */
#include <xinu.h>
void    operar(void), incrementar(void);
unsigned char x = 0;
```

```
/*------------------------------------------------------------------------
 * mut  --  programa con regiones criticas
 *------------------------------------------------------------------------
 */
void    mut(void)
{
        int i;

        resume( create(operar, 1024, 20, "process 1", 0) );
        resume( create(incrementar, 1024, 20, "process 2", 0) );

        sleep(10);

}


/*------------------------------------------------------------------------
 * operar x e y
 *------------------------------------------------------------------------
 */
void    operar(void)
{
        int y = 0;

        printf("Si no existen mensajes de ERROR entonces todo va OK! \n");

        while (1) {

                /* si x es multiplo de 10 */
                if ((x % 10) == 0) {

                        y = x * 2;      /* como y es el doble de x entonces
                                         * y es multiplo de 10 tambien
                                         */

                        /* si y no es multiplo de 10 entonces hubo un error */
                        if ((y % 10) != 0)
                                printf("\r ERROR!! y=%d, x=%d \r", y, x);
                }
        }
}

/*------------------------------------------------------------------------
 * incrementar x
 *------------------------------------------------------------------------
 */
void    incrementar(void)
{
        while (1) {
                x = x + 1;
        }
}
```

Ejecutar el programa. ¿Por qué este programa (sus procesos) emiten mensajes de error? ¿Qué es posible hacer para solucionar el problema?.

b. El sistema operativo Xinu no provee mecanismos para resguardar regiones críticas. Pero, su autor, define a Xinu como un microkernel, lo que significa que provee mecanismos mínimos que permiten luego construir otros más complejos. Usando el mecanismo de semáforos provisto por Xinu, implementar una protección de exclusión mutua (mutex). Este mutex debe estar compuesto por dos funciones :
**mutex_init(), mutex_lock(); y mutex_unlock();**

c. Proteger las regiones críticas del programa original con este nuevo mecanismo de exclusión mutua.